

```

import pandas as pd
import gurobipy as gp
from gurobipy import GRB

# Values that are variable and change

INPUT_CSV      = "battery_dataset_20260327_cleaned.csv"
ACCOUNT_NUMBER = "account_b"
OUTPUT_CSV     = "battery_schedule_AcctB_1battery.csv"

NUM_BATTERIES  = 1      # Change this to model different quantities
CAPACITY_KWH   = 13.5   # Max energy stored per battery
CHARGE_RATE_KW = 5.0    # Max charge power per battery
DISCHARGE_RATE_KW = 11.5 # Max discharge power per battery
CHARGE_EFF     = 0.9434 # Charging efficiency
DISCHARGE_EFF  = 0.9434 # Discharging efficiency
RESERVE_KWH    = 0      # Minimum SOC reserve per battery – battery will never discharge below this
                                # Set to 0 for no reserve

# RTE = 0.9434 × 0.9434 ≈ 89%

# Data Loading

df_raw = pd.read_csv(INPUT_CSV)
df = df_raw[df_raw["AccountNumber"] == ACCOUNT_NUMBER].copy()

df["Datetime"] = pd.to_datetime(df["Datetime"], format="mixed", errors="coerce")
df = df.dropna(subset=["Datetime"])

df["date_only"] = df["Datetime"].dt.date
df["hour"]      = df["Datetime"].dt.hour

# Assign integer day indices (number each unique date)
all_days = sorted(df["date_only"].unique())
day_map = {day: i + 1 for i, day in enumerate(all_days)}
df["d"]  = df["date_only"].map(day_map)
df["t"]  = df["hour"]

# One row per day + hour
hourly = df[["d", "t", "Quantity", "Price"]].copy()

# lookup dictionaries
Price = {(int(r.d), int(r.t)): float(r.Price) for r in hourly.itertuples()}
Quantity = {(int(r.d), int(r.t)): float(r.Quantity) for r in hourly.itertuples()}

DT = sorted(Quantity.keys()) # all day and hour pairs sorted in order

# Battery parameters

total_capacity = CAPACITY_KWH * NUM_BATTERIES
total_charge_rate = CHARGE_RATE_KW * NUM_BATTERIES
total_discharge_rate = DISCHARGE_RATE_KW * NUM_BATTERIES
total_reserve = RESERVE_KWH * NUM_BATTERIES

# Baseline cost
baseline_cost = sum(Quantity[k] * Price[k] for k in DT)

# OPTIMIZATION MODEL

m = gp.Model("battery_dispatch")
m.Params.OutputFlag = 1

# Decision variables
# grid[d,t] is energy purchased from grid
# charge[d,t] is energy sent into battery
# discharge[d,t] is energy drawn from battery
# soc[d,t] is battery state of charge at end of hour
# z[d,t] is 1 = charging this hour, 0 = discharging

grid = m.addVars(DT, lb=0, vtype=GRB.CONTINUOUS, name="grid")
charge = m.addVars(DT, lb=0, vtype=GRB.CONTINUOUS, name="charge")
discharge = m.addVars(DT, lb=0, vtype=GRB.CONTINUOUS, name="discharge")
soc = m.addVars(DT, lb=0, ub=total_capacity, vtype=GRB.CONTINUOUS, name="soc")
z = m.addVars(DT, vtype=GRB.BINARY, name="mode")

```

```

# Constraints

for d, t in DT:
    # Energy balance constraint where every kwh of demand must come from the grid or the battery
    m.addConstr(
        grid[d, t] + discharge[d, t] == Quantity[d, t] + charge[d, t],
        name=f"balance_{d}_{t}"
    )
    # Can only charge or discharge in a given hour, not both
    m.addConstr(charge[d, t] <= total_charge_rate * z[d, t], name=f"ch_limit_{d}_{t}")
    m.addConstr(discharge[d, t] <= total_discharge_rate * (1 - z[d, t]), name=f"dis_limit_{d}_{t}")

    # Battery never discharges below the customer's emergency reserve
    m.addConstr(soc[d, t] >= total_reserve, name=f"reserve_{d}_{t}")

for i, (d, t) in enumerate(DT):
    if i == 0:
        # Battery starts empty
        m.addConstr(
            soc[d, t] == CHARGE_EFF * charge[d, t] - discharge[d, t] / DISCHARGE_EFF,
            name=f"soc_init_{d}_{t}"
        )
    else:
        prev_d, prev_t = DT[i - 1]
        # SOC this hour = SOC last hour + what went in - what came out
        m.addConstr(
            soc[d, t] == soc[prev_d, prev_t]
                + CHARGE_EFF * charge[d, t]
                - discharge[d, t] / DISCHARGE_EFF,
            name=f"soc_flow_{d}_{t}"
        )

# Battery ends empty
last_d, last_t = DT[-1]
m.addConstr(soc[last_d, last_t] == 0, name="end_empty")

# Objective
m.setObjective(
    gp.quicksum(grid[d, t] * Price[d, t] for d, t in DT),
    GRB.MINIMIZE
)

m.optimize()

# Results

if m.status != GRB.OPTIMAL:
    print("No optimal solution found. Gurobi status:", m.status)
else:
    optimized_cost = sum(grid[d, t].X * Price[d, t] for d, t in DT)
    savings = baseline_cost - optimized_cost

    print("OPTIMAL SOLUTION")
    print(f" Account      : {ACCOUNT_NUMBER}")
    print(f" Batteries     : {NUM_BATTERIES} ({total_capacity:.1f} kWh total)")
    print(f" Reserve       : {total_reserve:.2f} kWh")
    print(f" Start SOC      : 0.00 kWh (empty)")
    print(f" End SOC        : 0.00 kWh (empty)")
    print(f" Baseline cost  : ${baseline_cost:>12,.2f}")
    print(f" Optimized cost : ${optimized_cost:>12,.2f}")
    print(f" Annual savings : ${savings:>12,.2f}")

    # Print hourly activity – only hours where the battery does something
    print("\nHourly battery activity:")
    print(f"{'Day':>5} {'Hr':>4} {'Price':>4} {'Grid':>8} {'Charge':>8} {'Discharge':>9} {'SOC':>8} Action")
    for d, t in DT:
        ch = charge[d, t].X
        dis = discharge[d, t].X
        if ch <= 1e-6 and dis <= 1e-6:
            continue
        action = "CHARGE" if ch > 1e-6 else "DISCHARGE"
        print(
            f"{d:>5} {t:>4} {Price[d,t]:>8.4f} {grid[d,t].X:>8.4f}"
            f" {ch:>8.4f} {dis:>9.4f} {soc[d,t].X:>8.4f} {action}"
        )

# Export schedule

```

```

reverse_day_map = {v: k for k, v in day_map.items()}

rows = []
for d, t in DT:
    ch = charge[d, t].X
    dis = discharge[d, t].X
    if ch <= 1e-6 and dis <= 1e-6:
        continue

    actual_dt      = pd.to_datetime(reverse_day_map[d]) + pd.Timedelta(hours=int(t))
    g              = grid[d, t].X
    s              = soc[d, t].X
    p              = Price[d, t]
    demand         = Quantity[d, t]
    hourly_cost   = g * p
    original_cost_hour = demand * p

    rows.append({
        "Datetime"      : actual_dt,
        "Price"          : p,
        "Demand_kWh"     : demand,
        "Grid_kWh"       : g,
        "Charge_kWh"     : ch,
        "Discharge_kWh"  : dis,
        "SOC_kWh"        : s,
        "Action"         : "CHARGE" if ch > 1e-6 else "DISCHARGE",
        "Net_Battery_kWh" : dis - ch,
        "Grid_Cost"      : hourly_cost,
        "Baseline_Cost"  : original_cost_hour,
        "Hourly_Savings" : original_cost_hour - hourly_cost,
    })

schedule_df = (
    pd.DataFrame(rows)
    .sort_values("Datetime")
    .reset_index(drop=True)
    .round(4)
)

schedule_df.to_csv(OUTPUT_CSV, index=False)

```